

pyfs-matrix cheatsheet

Typical flow: inventory (new mesh) → plan (mandatory, no solver) → run (--resume to add points) → collect (cluster jobs) → post | Safe by default: storage, sync, restore, rebuild and mark-failed only PREVIEW until --apply. --workspace defaults to the current folder; --runs NAME picks another manifest than runs.json; a matrix name is found in the workspace root or inputs/matrices/.

1. PREPARE THE WORKSPACE AND THE MATRIX

inventory <geometry> write <stem>.boundaries.toml beside a saved simulation (.fsm, from its mesh block) or an OBJ (from its groups); names the solver actions saved in a .fsm; no executable needed

- overwrite rewrite an existing .fsm sidecar (an OBJ sidecar is never rewritten)
- clean remove the saved solver actions (26.120 in meters: also reset to a fresh import, meshes and BCs kept); copy kept as <file>.bak-<stamp>

degenerate <geometry> derive a degenerate geometry, the thin blade first, from a blade mesh (.fsm or OBJ) and write <stem>.thin_blade.obj and its .boundaries.toml beside it; no executable needed

- root-offset LENGTH required: the root moved outward along the span, in the mesh's unit, off the spinner
- kind thin-blade the one kind (the default)
- boundary NAME the blade's boundary, for a file that also holds a spinner or a nacelle
- overwrite rewrite an existing output

audit-mesh <OBJ> audit a panel mesh (topology, trailing edge, panel quality) and write <stem>.audit.json beside it; exits 0 pass, 1 a gate or check failed, 2 refused

- against SOURCE judge against the mesh it was made from (alone: G1, G2, G4 only)
- csv FILE also write the figures, one row per family and figure

refine <MESH> [FACTOR] refine or coarsen a panel mesh from its OBJ into a new geometry folder <stem>.<tag>/ (OBJ, trailing-edge points, boundaries, refine.json, audit); FACTOR multiplies every selected family, else the factors come from the refinement file; the source is never modified

- families A,B only these families (default: all)
- chordwise F, --spanwise F the factor in one index direction of the grid families
- config FILE the refinement file (default: <stem>.refine.toml beside the mesh)
- out-dir DIR where the level folder goes (default: the parent of the source's folder)
- overwrite replace an existing level folder

upgrade <matrix> convert a matrix written under an older layout

- in-place rewrite the file (default: print to stdout)
- inputs DIR also move inputs/groups/*.toml into pproc [groups]; needs --in-place

rename rename a 0.20.x workspace to the 0.21.0 point names (folders, scripts, manifest, plan)

- workspace DIR --dry-run print what would move, change nothing

convert <matrix> emit the equivalent native campaign.toml

- fs-exe PATH required: FlightStream executable (never guessed)
- name NAME campaign name (required here: no workspace)
- fs-version V build for rows with empty FS_BUILD, e.g. 26.124
- recipe CODE=MODULE:FUNCTION FS_SCRIPT code to recipe (repeatable)
- o, --output FILE write here instead of stdout

2. PLAN (MANDATORY BEFORE RUN, NO SOLVER)

plan <matrix> bind the matrix to inputs/, pre-flight every point, write the receipt run requires (pinned to the matrix digest; edit the matrix, plan again). Warns, never refuses, on saved actions in an unsteady geometry and on families a row lacks. Alias: inspect-setup

- workspace DIR campaign root with inputs/
- name NAME campaign name (default: workspace folder name)
- fs-version V fallback build for empty FS_BUILD cells
- fs-exe PATH executable override; mandatory for MANUAL rows (else FS_BUILD via inputs/executables.toml)
- workflow CODE=NAME FS_SCRIPT code to run type (repeatable)
- recipe CODE=MODULE:FUNCTION LEGACY recipe (repeatable)
- point-name TPL naming template; default P<sim>~<point name>; placeholders {point} {alpha} {beta} {mach} {advance_ratio} {sim} {campaign}
- ignore-missing-families [true|false] default true: a family the mesh lacks is skipped; false refuses the point

- no-ignore-missing-families same as false
- cost table mesh size, TEs, farfield layers, viscous coupling, steady/unsteady, iterations, CPUs and an EXPECTED wall time fitted on this workspace's runs
- cost-file NAME with --cost or a grouped plan: price the points by inputs/costs/NAME.toml, the machine's own cost model (mesh, steps, CPUs, flags); needed only when several exist
- inflow-fft for qsteady_rotor wheel points in a custom inflow: blade-frame harmonics, k_{eff} , n_{max} ; suggested PASSAGE_POSITIONS (warns if too few)
- update-ids (~updateIDs) REWRITE this matrix so no POL repeats one used elsewhere; written before the plan runs
- accept-unregistered-build allow an installed build other than the registered one (your responsibility; recorded)
- setup-guidelines write inputs/setup/SETUP_GUIDELINES.md
- setup-standards write commented s9XX setup standards
- sims SIM [SIM ...] plan only these simulations, e.g. --sims 2031 2032
- points POINT [POINT ...] with --sims: only these points, by the point name the plan prints; the matrix is not edited
- polar-sweep plan each simulation as ONE solver job; steady or unsteady polars, never both in a job; with --local
- batch N plan the simulations as N solver jobs; steady or unsteady polars, never both in a job; not with --polar-sweep
- hpc NAME use inputs/hpc/NAME.toml when the workspace holds several profiles
- verbose full warnings (all detail also in logs/activity.log)

3. RUN

run <matrix> run every active point and write the sweep table. Needs a current plan. Linux with a submission profile submits to the cluster; Windows runs locally

- resume skip points already recorded (a grown matrix runs only new points); without it a recorded point refuses
- force-rerun POINT REDO a recorded point (by point name, run_id or steady JOB id; repeatable). Old records and outputs are archived, nothing deleted. Not with --resume
- force-rerun-all REDO every recorded point (prints the license count first)
- sims SIM [SIM ...] run only these simulations, space separated, e.g. --sims 2031 2032; with --force-rerun-all, redo only these
- points POINT [POINT ...] with --sims: run only these points, by the point name the plan prints (not with --force-rerun-all)
- local run on THIS machine instead of submitting (Linux workstation or smoke test)
- polar-sweep run each simulation as ONE solver job; steady or unsteady polars, never both in a job; with --local
- batch N run the simulations as N solver jobs; steady or unsteady polars, never both in a job; not with --polar-sweep
- hpc NAME use inputs/hpc/NAME.toml when the workspace holds several profiles
- progress-every N local unsteady progress every N steps (default 10; 0 = silent)
- sweep-csv FILE sweep table path (default post/<matrix>/campaign_sweep.csv)
- workflow CODE=NAME run type: steady, unsteady, unsteady_rotor, qsteady_rotor
- workspace, --name, --fs-version, --fs-exe, --recipe, --point-name, --ignore-missing-families,
- no-ignore-missing-families, --accept-unregistered-build: as in plan
- pproc-warnings print grouped post-processing warnings
- verbose full warnings and one line per repeated item

4. COLLECT (CLUSTER OR INTERRUPTED RUNS)

collect for every SUBMITTED record, wait until its outputs are present and SETTLED, collect, assess, update the record, then post; HPC job_end_files present, no solver log: FAILED_EXECUTION

- watch keep sweeping until nothing is outstanding (default: one sweep)
- discard-walltime mark clock-stopped points failed before post; grouped plans rerun them from the start, default-mode plans need an explicit rerun; keep outputs
- watch-interval S seconds between sweeps (default 60)
- interval S seconds between the two size/time observations (default 2)
- sims N stop a watch after N sweeps
- runs IDS only these simulations, e.g. 2006,2007, and their post
- no-post collect only, do not rebuild products

pyflightstream 0.38.0 every subcommand and option, by stage pyfs-matrix <cmd> -h (--help) gives the full text

- hpc NAME use inputs/hpc/NAME.toml when the workspace holds several profiles
- check-frozen REFUSE (not warn) averages touched by a frozen solve or an unread log block
- workspace, --runs NAME, --pproc-warnings, --verbose

Tip: nohup pyfs-matrix collect --watch > logs/collect.out 2>&1 & status one row per polar: SIM, POLAR, SWEEP, POINTS (recorded/planned), MATRIX, STATUS (one word, or the count of each); a point no record carries reads planned; reads only, writes nothing

- sims 2006,2007 (or [2006,2007]) --matrix STEM --status WORD (repeatable, FAILED,*) --failed
- additional lists the additional register; --points one row per datapoint --json --csv --workspace, --runs NAME

show <run id or alias> outcome first, then evidence tails, logs, continuation chain, identity and inputs; coupled points also show the FSI record and last exchange row

- pyfs-matrix show 9101_1; a simulation id alone lists its points and failed cards; --point NAME selects one
- workspace, --runs NAME, --json, --csv

log read recorded activity oldest first, including running batches and compacted simulations

- pyfs-matrix log --sims IDS --run ID --stage NAME --since 2h --problems --open
- post MATRIX: groups by category, product family and message shape, with counts and examples; --sims IDS narrows them
- storage: storage calls as recorded; --stage selects the recorded action
- workspace, --runs NAME, --json, --csv; --since also accepts an ISO time

trace <product path> follow the products index to the simulation, post options, effective run identities, digests and provenance paths

- pyfs-matrix trace post/demo/polars/polar.csv; or --run ID for the provenance tree (aliases accepted)
- workspace, --runs NAME, --json, --csv

pyfs-matrix status --additional prints the additional register's fields as recorded. These queries write nothing, take no lock and never ask the scheduler. history <simulation or point> pyfs-matrix history reads every present and archived record, its continuation and archive stamp; batch folders and compacted simulations are read in place

- workspace DIR --runs NAME --json --csv

diff <run-a> <run-b> pyfs-matrix diff compares versions, digests, solver flags, flight condition and outcome; run-id@<archive stamp> selects an archived copy; script differences name lines and inputs

- workspace DIR --runs NAME --json --csv; reads only, writes nothing

5. POST-PROCESS (NO SOLVER, NO SEAT)

post [matrix] rebuild polars, sections and plot tables under post/<matrix>; no matrix = every matrix in the manifest. Old products are archived in post/<matrix>/archive/

- sims IDS only these simulations' products, in place (super files kept)
- runs NAME read another manifest (e.g. a rebuilt one)
- from-sims ignore runs.json, build records in memory from sims/ (runs made outside the package); products go to post/<matrix>@sims/
- steps-per-revolution N with --from-sims: for a LAST_REVS_AVG window
- additional-pproc reopen the final .fsm of rows with ADDITIONAL_PPROC (no solve, one launch per point) and extract that pproc; 26.124 only. With it: --fs-version,
- fs-exe, --local, --recipe
- diagnostics print recorded post diagnostics as Markdown, change nothing
- strict exit 3 when a product was skipped by design (e.g. polar under sideslip)
- check-frozen refuse instead of warn (see collect)
- archive keep the replaced products under archive/ (default: overwrite in place)
- force-overwrite DESTROY products instead of archiving (asks; --yes answers it)
- workspace, --pproc-warnings, --verbose

6. SYNC WORKSPACES (E.G. CLUSTER TO WINDOWS)

sync {runs|post|fsm|all} run on the MAIN workspace; peers in inputs/sync-workspaces.toml; levels cumulative, runs < post < fsm < all. Nothing in main is deleted; inputs/ untouched but a declared matrix

- apply change files (default: preview)
- from NAME one peer (default: every non-main one)
- prefer-other on a record conflict take the peer's (default: main wins)
- overwrite on a file conflict archive main's copy and take the peer's
- restore with --apply: rebuild records of sims/ folders no record carries
- include-archives also bring archive folders (skipped by default)
- workspace, --runs NAME

7. RECOVER AND MARK RECORDS

restore {runs|storage|products|plan|additional} restore a file from archive/

- apply (default: preview) --stamp STAMP archive to restore from
- matrix STEM for products and plan, when several matrices have archives
- workspace

rebuild rebuild run records from the folders under sims/

- apply (default: preview) --out NAME write records to another manifest (compare before replacing)
- all-sims every simulation folder, recorded or not --sims 4001,2009 only these
- matrix FILE the matrix the simulations ran from
- build-alias BUILD=ALIAS scheduler name of a build (repeatable), e.g. 26.124+26.12
- inputs-from FOLDER another origin's inputs/ for inputs changed after the run
- workspace

mark-failed every record of the named simulations becomes FAILED_MARKED, its old status kept under marked; runs.json archived first. Not readable before 0.33.0

- sims 2006,2007 the simulations --reason TEXT why, recorded as given
- apply (default: preview) --workspace

mark-converged the named points become CONVERGED by your reading, their old status kept under marked; runs.json archived first

- sims 2006 2007 --points AL+O20 only these points --reason TEXT --apply --workspace

8. DISK SPACE (EVERY CALL LOGGED IN storage_management.json)

space-in-use sizes by top folder, by sims/sim_*, by extension; changes nothing

- top N rows per grouping (default 15) --workspace

free-space <m-id> run storage recipe inputs/management/m<id>.toml: prune_step_exports, compact_sims (zip, restored automatically), delete_extensions, post_archives

- apply (default: preview) --runs NAME --workspace
- list every path each step touches, with its size and what happens to it

delete-sims <ids> delete simulations, their products and records, e.g. 4001,2009; a note row remains

- apply (default: preview) --runs NAME --workspace
- force also a simulation whose run is still SUBMITTED (refused without it)
- matrix-products {points-only|regenerate} for a matrix product that also holds the deleted points: leave it (marked stale) or rerun its post without them

Before a licensed run: plan with --cost; read the warnings; inventory --clean a geometry that carries saved actions. Exit codes of post: 0 done; 2 refused, nothing written; 3 with --strict, a product skipped by design. Trace: runs.json, plan.json, post/, logs/.

Every other command-line tool

The tools: pyfs-workspace builds the campaign tree and custom free-stream fields; pyfs-qa is the evidence tooling of the command database (probes and physics runs need a licensed machine); pyfs-fsi is the coupling executable the solver calls; pyfs-manual is a maintainer tool (the [manual] extra); the two python -m entries keep an optional workbook (the [excel] extra). Safe by default: the field operations preview until --apply; pyfs-manual writes only with --write.

pyfs-workspace

the campaign tree, the custom free-stream fields of inputs/freestreams/ and the disc profiles

init [root] create or complete the campaign tree: inputs/ with its library folders, sims/, post/, archive/; writes an inputs/executables.toml template when none exists. Default root: the current folder

archive <root> <sim_id> zip one recorded simulation under archive/ and remove its folder sims/sim_<sim_id>; a simulation the manifest does not record is refused

migrate-geometries [root] move a flat inputs/geometries/ library into one folder per geometry

field build a custom free-stream file of inputs/freestreams/ from other fields. Every operation below previews until --apply, writes the file <STEM> (.txt STRUCTURED, .dat UNSTRUCTURED, the form decides) and its provenance record, and takes --out STEM (required), --workspace DIR, --apply, --overwrite (replace an existing <STEM> file and its record)

field mirror <field> mirror a field through the plane $x = 0$, $y = 0$ or $z = 0$

- plane {x,y,z} required: the coordinate whose sign changes
- out STEM --workspace --apply --overwrite

field move <field> translate a field so a source point lands on a target point

- source-point X Y Z required: the source point (a hub), m, global frame
- target-point X Y Z required: where it lands, m, global frame
- out STEM --workspace --apply --overwrite

field subtract <total> <other> total minus (other minus reference), point by point on one grid

- reference VX VY VZ required: the free stream OTHER was solved in, m/s (0 0 0 only where OTHER is already induced only)
- tolerance M two positions closer than this, per axis, are one point (default 1e-06)
- out STEM --workspace --apply --overwrite

field time-mean <files> the time mean of an unsteady run's per-step fields (files or glob patterns)

- last K average only the last K steps given (default: every step)
- fluctuation also write <stem>.fluctuation.csv, the per-probe standard deviation (two steps or more)
- fluctuation-only report the fluctuation only, no field (needs --last)
- vinf M_S free-stream speed, m/s, the fluctuation summary is also stated against
- tolerance M as in subtract
- out STEM --workspace --apply --overwrite

field fill-interior <field> fill the probes inside the body from the ray outside it

- r-body M body radius about the x axis, m (default 0.38)
- out STEM --workspace --apply --overwrite

profile <shape> [table] the actuator-disc profile a row's PROFILE names, inputs/profiles/<STEM>.csv, scaled to --thrust or --ct --rho --rpm; ELLIPTICAL needs none. sections: --pol --family --component --sign --last; bp: --advance-ratio; uni, bp: --stations; disc: --ref --disc, or --tip-radius --hub-radius --blades; --out --workspace --apply --overwrite

pyfs-fsi

the coupling executable the solver's aeroelastic toolbox calls once per time step. With no arguments it runs one coupling step in the current folder: coupled when config.json is there, the dummy (zero displacements) otherwise

init-dummy write the dummy configuration into a run folder

- node-count N required --dir DIR run folder (default: current)

step execute one coupling step explicitly

- dir DIR run folder (default: current)

pyfs-qa

evidence tooling of the command database: probes run the solver locally on a licensed machine

probe run command-validity probes and write the compat report pair

- fs-version V required: canonical build, e.g. 26.120
- fs-exe PATH required: the executable (never guessed)
- commands LIST comma-separated subset (default: every command with a probe spec)
- identity-only judge no command: capture the solver's identity banner, the way a new build gets its number into a committed report
- fsm FILE local saved simulation for preludes that need one
- workroot DIR scratch root (default runs/probes)
- timeout S per-probe limit, seconds (default 120)
- report-dir DIR (default reports/compat) --label TEXT report stem suffix

apply-compat <report> promote database statuses from a committed compat report

- root DIR repository root the report is cited against (default .)

physics run the physics matrix of a workspace, reduce its records, write the physics report

- workspace DIR workspace with the physics matrix at its root (default: current folder)
- matrix FILE (default matriz_physics.fs) --name NAME campaign name
- resume skip points already recorded
- report-dir DIR (default reports/physics) --label TEXT

drift run the physics matrix on two builds, each under its own registry, and diff the reductions

- fs-versions A,B required: baseline first
- fs-exe VERSION=PATH required, repeatable: one executable per version
- workroot DIR where the two sides are made (default runs/drift); a side that exists is refused
- workspace, --matrix, --name, --report-dir, --label: as in physics

cases print the Tier 3 test matrix, one line per case id; nothing runs

- include-smi also the local-geometry class, never listed implicitly

cost <campaign> wall-time cost per simulation and point from runs.json, one column per build; runs nothing

- compare BASELINE,CANDIDATE two build columns, baseline first

update-reference <case> update or seed one physics reference from a committed physics report

- from-report FILE required --reason TEXT required, recorded in the reference

pyflightstream 0.38.0 every subcommand and option <tool> <cmd> -h (--help) gives the full text

pyfs-manual

maintainer tool, the [manual] extra: compares a vendor manual with the command database. The manual path is passed explicitly and nothing of it is copied

coverage what the manual documents and the database does not

- manual PDF --source ID --chapter-pages FIRST-LAST all required
- index-pages FIRST-LAST the Script Index (optional)

draft render database entries for the absent commands, for review

- manual, --source, --chapter-pages, --index-pages: as in coverage
- fs-version CANONICAL required, repeatable
- only TEXT filter on the command name --out FILE destination
- write actually write (default: print, touch nothing)

sweep read every edition; report what no entry covers

- editions MANIFEST required: YAML, one row per build
- by-section group by section --fail-if-absent exit 1 when a documented command cannot be emitted

surface the command surface of each edition and what changed between builds

- editions MANIFEST required, in release order
- names list the names, not only counts --markdown render as Markdown

citations re-read the database's page citations against the manuals

- editions MANIFEST required

register write documented rows for a new edition where the documentation did not change

- editions MANIFEST required --fs-version CANONICAL required
- commands-dir DIR chapter files to edit (default: the installed package)
- write actually write (default: print)

python -m pyflightstream.workspace.excel

the optional macro-free workbook, the [excel] extra; save and close it first

create <output> create a new workbook

- workspace DIR required

preview <workbook> inspect changes without applying them (a batch JSON and its HTML)

- workspace DIR required --batch FILE required
- direction {read,write} required: read imports matrices, write exports
- matrix FILE select a subset (repeatable)

apply <batch> apply one fresh reviewed batch (single use)

cancel <batch> cancel a preview without changing inputs

check <workbook> validate its Runs and Dictionary

python -m pyflightstream.workspace.excel_bridge

the text bridge the workbook's own controls call: <action> is preview, apply or check; --request FILE, --response FILE and --batch FILE are all required

Copyright (c) 2026 Geovana Neves, CC BY 4.0.

https://github.com/nevesgeovana/pyflightstream

1. The workspace and its input library

A workspace is the folder the package owns. The input library is what you bring, staged once and cited by identifier; the run records are what the package writes. Folder names are never authoritative.

pyflightstream 0.38.0 the campaign workflow by stage, stage 1 of 8



What it is for

Create the tree once, stage each geometry under `inputs/geometries/`, write its boundary inventory, and build any custom free-stream field the study needs. Every later stage cites these files by identifier (a geometry by its file name, a reference, setup or pproc by its coded id), never by a path.

Figure: the workspace tree

```
<workspace>/
├── inputs/ the input library
│   ├── geometries/ .fsm, .obj, .stl and <stem>.boundaries.toml
│   ├── references/ setups/ pproc/ r####, s####, p#### (stage 3)
│   ├── profiles/ freestreams/ custom free-stream fields
│   ├── calibrations/ fsi/ structural inputs
│   ├── hpc/ one submission profile (stage 5)
│   ├── matrices/ a home of the run matrices
│   ├── management/ storage recipes m<id>.toml
│   ├── executables.toml the build registry
│   └── INPUTS.md input_template.md generated guides
├── sims/sim_POL/ one folder per simulation
│   ├── inputs/ scripts/ staged inputs, solver scripts
│   └── datapoints/DP-<point>/ one folder per point
├── post/<matrix>/ plan receipt and products
├── archive/ archived records and simulations
├── logs/activity.log every command's log
├── runs.json the run records
└── storage_management.json every storage call
```

Commands

pyfs-workspace init [root]

Create or complete the tree: every `inputs/` folder, `sims/`, `post/`, `archive/`, an `inputs/executables.toml` template when none exists, and the generated guides. Safe to run again; the default root is the current folder.

pyfs-workspace migrate-geometries [root]

Move a flat `inputs/geometries/` into one folder per geometry, the layout a new workspace uses.

pyfs-matrix inventory <geometry>

Write `<stem>.boundaries.toml` beside a saved simulation (from its mesh block) or an OBJ (from its groups), and name the solver actions a saved simulation keeps. Needs no executable.

- overwrite rewrite a saved simulation's sidecar that already exists.
- clean remove the saved solver actions, keeping <file>.bak-<stamp> (stage 8).

pyfs-matrix degenerate <geometry>

Derive the thin blade of a blade mesh (a saved simulation or an OBJ): the sheet midway between its two sides, written beside the source with its boundary inventory. Needs no executable.

- root-offset LENGTH required: how far the root moves outward along the span, in the mesh's unit.
- kind thin-blade the one kind, the default.
- boundary NAME the blade's boundary, for a file that also holds a spinner or a nacelle.
- overwrite rewrite an existing output.

Custom free-stream fields

Each operation writes one file of `inputs/freestreams/`, named by --out STEM (.txt STRUCTURED, .dat UNSTRUCTURED, the form decides), with its provenance record. Each previews until --apply.

pyfs-workspace field mirror <field>

Mirror a field through the plane $x = 0$, $y = 0$ or $z = 0$.

- plane {x,y,z} required: the coordinate whose sign changes.
- out STEM required: the stem of the file written.
- workspace DIR --apply writes (default: preview); --overwrite replaces an existing file and its record.

pyfs-workspace field move <field>

Translate a field so a source point lands on a target point.

- source-point X Y Z required: the source point (a hub), m, global frame.
- target-point X Y Z required: where it lands, m, global frame.
- out STEM --workspace, --apply, --overwrite: as in mirror.

pyfs-workspace field subtract <total> <other>

Total minus (other minus reference), point by point on one grid.

- reference VX VY VZ required: the free stream OTHER was solved in, m/s.
- tolerance M positions closer than this per axis are one point (default 1e-06).
- out STEM --workspace, --apply, --overwrite: as in mirror.

pyfs-workspace field time-mean <files>

The time mean of an unsteady run's per-step fields (files or glob patterns).

- last K average only the last K steps given.
- fluctuation also write <stem>.fluctuation.csv, the per-probe standard deviation.
- fluctuation-only the fluctuation report alone, no field; needs --last.
- vinf M_S the free-stream speed the fluctuation is also stated against.
- tolerance M as in subtract.
- out STEM --workspace, --apply, --overwrite: as in mirror.

pyfs-workspace field fill-interior <field>

Fill the probes inside the body from the ray outside it.

- r-body M the body radius about the x axis, m (default 0.38).
- out STEM --workspace, --apply, --overwrite: as in mirror.

Actuator-disc profiles

pyfs-workspace profile sections (--pol), uni or bp writes `inputs/profiles/<STEM>.csv`, a row's PROFILE, scaled to --thrust or --ct. ELLIPTICAL needs no file.

Mesh levels

pyfs-matrix refine <MESH> [FACTOR]

A level in a new geometry folder `<stem>_<tag>/` (OBJ, trailing-edge points, boundaries, refine.json, audit); the source is never modified. FACTOR multiplies every selected family.

- families A,B only these; --chordwise F, --spanwise F: one direction of the grid families.

--config FILE default <stem>.refine.toml: [families.<name>] factor, chordwise, spanwise, method, elements, axial, circumferential, axis, origin; [refine] tag, elements; [components]; [periodic].
--out-dir DIR default: beside the source's folder; --overwrite replaces a level.

pyfs-matrix audit-mesh <OBJ>

Writes <stem>.audit.json: G1, G2, G4 alone, G1 to G6 and relative checks with --against SOURCE; --csv FILE. Exit 0 passed, 1 failed, 2 refused.

Files

- R inputs/geometries/ the saved simulation or mesh pyfs-matrix inventory reads.
- W <stem>.boundaries.toml beside the geometry: the boundary names every alias and pproc family cites.
- W inputs/executables.toml the build registry template: each build a row may name, its executable.
- W inputs/geometries/README.md how to lay out the geometries.
- W inputs/INPUTS.md the glossary of every matrix column and input key.
- W inputs/input_template.md an example of every input file.
- W inputs/pproc/VARIABLES.md with WRITING-EQUATIONS.md: pproc variables and equations.
- W inputs/freestreams/<STEM> a custom free-stream field and its record.

Common errors

- ! pyfs-workspace init: an ambiguous artifact id, two files of geometries/ or profiles/ with one stem (wing.fsm, wing.stl). Fix: the id is the stem: rename or remove one. The tree is still completed.
- ! The inventory refuses: the sidecar already exists. Fix: --overwrite rewrites a saved simulation's sidecar; an OBJ's never: edit or remove it.
- ! A field operation printed what it would write; nothing changed. Fix: that was the preview: run it again with --apply.
- ! The field file <STEM> already exists. Fix: another --out, or --overwrite it and its record.
- ! --fluctuation-only refuses. Fix: it needs --last K: the fluctuation of the last K steps.
- ! A bare stem in a GEOMETRY key is refused. Fix: name the file, extension included (wing_clean.fsm).

Where to read more

Documentation: *The workspace and the workflow*, *The input library and the solver presets*, *Field operations*; guide 1, *Workspaces*.

2. The run matrix and its two homes

The run matrix is the study: one row per polar, each naming its geometry, its three input artifacts, its flight condition and what it sweeps. The row says what is different; the library says what is shared.



What it is for

A matrix is a pipe-delimited text file, `<name>.fs`, one row per simulation. POL becomes the simulation id (`sim/sim_<POL>/`). FLIGHT_CONDITION states the flow as KEY:value pairs, and exactly one key carries the word `sweep`; SWEEP_VALUES lists its values, so one row is several points. - in a column states nothing.

The columns, in order

POL	HIDDEN RUN	id; solver window shown; does the row take part
AIRCRAFT CONFIGURATION	DESCRIPTION	what the row is (labels only)
FLIGHT_CONDITION	DESCRIPTION	your words
SWEEP_VALUES	DESCRIPTION	the flow, and what is swept
GEOMETRY	DESCRIPTION	the swept values
REF SET PPROC	DESCRIPTION	the mesh file, by its file name
SYMMETRY SYMMETRY_LOADS	DESCRIPTION	the coded ids <code>r###</code> , <code>s###</code> , <code>p###</code>
NCPUS WALLTIME	DESCRIPTION	what was meshed, which loads come back
FS_BUILD	DESCRIPTION	processors; wall clock with its unit (4h)
WORKFLOW	DESCRIPTION	the build the row wants, e.g. 26.123
VAR_NAMES_VALUES	DESCRIPTION	the run type: <code>steady</code> , <code>unsteady</code> , <code>unsteady_rotor</code> , <code>qsteady_rotor</code> , or <code>LEGACY</code> (a recipe)
	DESCRIPTION	every other key, KEY: value pairs separated by /

Reading one row

POL	7002: the simulation <code>sim/sim_7002/</code>
FLIGHT_CONDITION	TASmps:30.0, REMi:1.20, ALPHA:sweep
SWEEP_VALUES	0.0,2.0: two angles, two points
GEOMETRY	wing.fsm, staged in <code>inputs/geometries/</code>
REF SET PPROC	r003 s002 p001
FS_BUILD WORKFLOW	26.120 steady
VAR_NAMES_VALUES	VELOCITY: 30.0

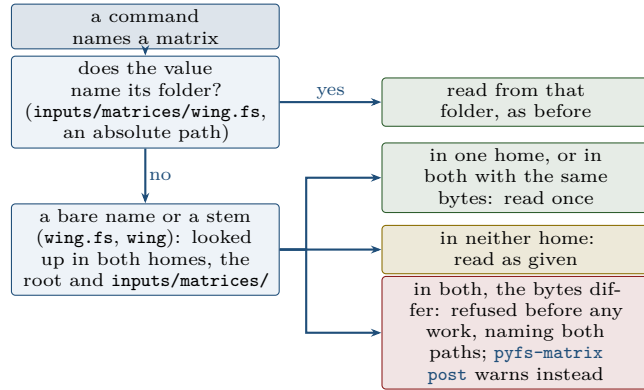
Several matrices in one workspace

Each matrix keeps its own `post/<matrix>/` folder, named after its file stem, with its own plan receipt and sweep table. A POL may be used by one row of one matrix only: `pyfs-matrix plan` finds a repeat, and its option to renumber is on stage 4.

How a point is named

The point name writes each flight-condition variable as a code and a fixed-width integer: M144RE438AL+000BE+000J+080. Scripts and exports are `P<sim>-<name>`, the run id is `<campaign>/sim_<POL>-<name>`, and a steady row is one job, `<campaign>/sim_<POL>/sweep`.

Figure: the two homes and the one lookup



Every command that takes a matrix reads it through this lookup: `pyfs-matrix upgrade`, `pyfs-matrix convert`, `pyfs-matrix plan`, `pyfs-matrix inspect-setups`, `pyfs-matrix run`, `pyfs-matrix post`, `pyfs-matrix rebuild` and `pyfs-matrix rename`. `pyfs-matrix upgrade` and `pyfs-matrix convert` take no workspace and look in the current folder.

Commands

`pyfs-matrix upgrade <matrix>`
Convert a matrix written under an older layout to the current one.
`--in-place` rewrite the file (default: print to standard output).
`--inputs DIR` also move `inputs/groups/e001.toml` to `inputs/pproc/p001.toml` under `[groups]`; needs `--in-place`.
`pyfs-matrix convert <matrix>`
Emit the native campaign.toml equivalent of a matrix.
`--fs-exe PATH` required: the executable, never guessed.
`--name NAME` the campaign name; required here, as there is no workspace.
`--fs-version V` the build for rows whose FS_BUILD cell is empty.
`--recipe CODE=MODULE:FUNCTION` a LEGACY recipe code (repeatable).
`-o, --output FILE` write here instead of standard output.
`pyfs-matrix rename`
Rename a workspace written under 0.20.x to the point names of 0.21.0 (folders, scripts, records, plan).
`--workspace DIR` the campaign root (default: current folder).
`--dry-run` print what would move and change nothing; read it first.

The optional workbook (the excel extra)

Save and close the workbook first. A read imports matrices into its Runs sheet, a write exports Runs to matrices; both go through a preview, then an apply.

```
python -m pyflightstream.workspace.excel create <output>
```

Create a new macro-free workbook.

`--workspace DIR` required: the campaign it belongs to.

```
python -m pyflightstream.workspace.excel preview <workbook>
```

Write a batch, its JSON and an HTML beside it; change nothing.

`--workspace DIR` required.
`--direction {read,write}` required: read imports, write exports.
`--batch FILE` required: the pending transaction; keep it unchanged.
`--matrix FILE` select a subset (repeatable).

```
python -m pyflightstream.workspace.excel apply <batch>
```

Apply one fresh reviewed batch; each batch is single use.

```
python -m pyflightstream.workspace.excel cancel <batch>
```

Cancel a preview; nothing changes.

```
python -m pyflightstream.workspace.excel check <workbook>
```

Validate its Runs and Dictionary.

Files

- R** `<name>.fs` in the workspace root or in `inputs/matrices/`, the two homes.
- W** `campaign.toml` written by `pyfs-matrix convert`.
- R** `inputs/sync-workspaces.toml` names the workspace that owns each matrix (stage 6).

Common errors

- ! One stem in both homes with different bytes: every command refuses, naming both paths.
Fix: keep one copy, or make the two identical.
- ! The plan warns that the matrix lies outside the workspace's matrix folders.
Fix: move it to the root or `inputs/matrices/`: `pyfs-matrix sync` and the repeated-POL check see only those two.
- ! A WALLTIME cell is a bare number.
Fix: write its unit: 240m, 4h, 90s, 1d.
- ! A fact is stated in its column and as a key of VAR_NAMES_VALUES.
Fix: refused naming both: keep one home for it.
- ! A row moved off LEGACY keeps an FSM_FILE key.
Fix: refused at plan naming the key: rename it to GEOMETRY.
- ! `--inputs` without `--in-place`.
Fix: the matrix cells that name the files are rewritten with them: add `--in-place`.

Copyright (c) 2026 Geovana Neves, CC BY 4.0.

<https://github.com/nevesgeovana/pyflightstream>. Every command and option: `<tool> <cmd> -h`, or pages 1 and 2 of this cheatsheet.

3. The reference, setup and pproc artifacts

Three small files a row cites by coded id: the reference **r###** (what the numbers are measured against and what the parts are called), the setup **s###** (how the solver runs) and the pproc **p###** (what is written down afterwards).

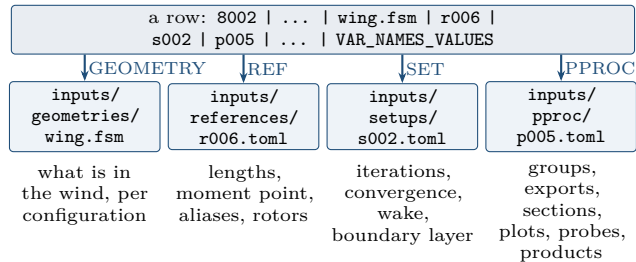
pyflightstream **0.38.0** the campaign workflow by stage, stage 3 of 8



What it is for

A study is many runs that share almost everything. What they share lives in these files, split by how often it changes, and a row names one of each: REF, SET and PPROC. The letter of a coded id declares its kind, so a mistyped number cannot reach another kind's file.

Figure: what a row resolves to



The reference r###

`area_m2`, `chord_m`, `span_m` and `[moment_point]` are what every coefficient is divided by and taken about. `[aliases]` names sets of boundaries once (`airframe` = `["Body", "Base"]`); a member the opened mesh lacks is left out, so one reference serves several geometries. A block with `kind` = "rotor" is a rotor: hub, axis, `rpm_sign`, `diameter_m`, its blade families; a row then names the rotor and nothing else about it.

A rotor block of the reference

```
[PORT]
kind = "rotor"
x_m = 4.5      # the hub; y_m, z_m likewise
axis = "X"     # what it turns about
rpm_sign = 1   # which way
diameter_m = 3.6576 # what J resolves against
families_blades = ["Blade1"] # the count is its length
```

Starting a library

Copy a working `inputs/` whole, delete the artifacts you do not need, edit the reference to your own lengths and boundary names, then write one matrix row and plan it.

The setup s###

The knobs set once for a study: iterations, convergence, the wake, the boundary-layer type, `solver_model`, `moments_model` (PRESSURE or VORTICITY). The time step is not here: it is what a point is, so it lives on the row. A row may state setup keys over its preset in `VAR_NAMES_VALUES`, under their native names, for that row only: the plan warns, a key the preset does not state or states equally is accepted, and the run record lists them under `from_row`.

The pproc p###

Boundary groups, exports, sectional distributions, force plots, probe lines, sampled fields and the product tables. The frame decides how an entry expands:

frame =	you get
MRP, or a declared frame	one, over the whole cited set
SMRP or RMRP	one per rotor, in its own frame
LOCAL_AXIS	one per blade, in its frame

A second artifact can be extracted after the run, with no solve: `ADDITIONAL_PPROC: p<id>` on the row and `pyfs-matrix post` with its additional post (stage 7).

Commands

These options of `pyfs-matrix plan` concern the artifacts; the rest of the plan is stage 4.

```
pyfs-matrix plan <matrix>
Bind the matrix to the library and pre-flight every point.
--setup-guidelines write inputs/setups/SETUP_GUIDELINES.md,
  keeping your edits.
--setup-standards write the commented s9XX setup standards,
  keeping your edits.
--ignore-missing-families [true|false] true (default): a family
  the opened mesh lacks is left out; false refuses the point.
--no-ignore-missing-families the same as false, as a flag.
```

Files

- R** `inputs/references/r006.toml` the reference a row's REF cell names.
- R** `inputs/setups/s002.toml` the setup preset of the SET cell.
- R** `inputs/pproc/p005.toml` the post-processing artifact of the PPROC cell.
- R** `<stem>.boundaries.toml` the boundary names aliases and families must match.
- W** `inputs/setups/SETUP_GUIDELINES.md` the setup guidance, on request.
- W** `inputs/pproc/VARIABLES.md` rewritten when a pproc gains a glossary.

Common errors

- !** A row states a setup key the preset states with another value.
Fix: refused at plan, naming both values: change the row or the preset.
- !** A rotor row's setup states `vorticity_drag_families` and `moments_model` = PRESSURE.
Fix: refused at plan: state VORTICITY or drop the drag list.
Unstated, a rotor row with the list takes VORTICITY and the plan warns.
- !** The plan warns that a section or plot group names a family a row's geometry lacks.
Fix: a misspelled family is named the same way: check the name against `<stem>.boundaries.toml`. The point plans;
`--ignore-missing-families false` refuses it instead.
- !** Both `--ignore-missing-families` and `--no-ignore-missing-families` are stated.
Fix: refused: state one.
- !** A coefficient looks off by a constant factor.
Fix: check `area_m2`, `chord_m` and `span_m` of the cited reference: every coefficient is divided by them.
- !** An unsteady row states no averaging window.
Fix: every unsteady row must state one: `LAST_REVS_AVG` in revolutions, or `LAST_ITERS_AVG` in time iterations.

Where to read more

Guide 3, *References*; guide 4, *Solver setup*; guide 5, *Post-processing definitions*; the documentation pages *The reference artifact* and *The post-processing artifact and its products*.
Copyright (c) 2026 Geovana Neves, CC BY 4.0.
<https://github.com/nevesgeovana/pyflightstream>. Every command and option: `<tool> <cmd> -h`, or pages 1 and 2 of this cheatsheet.

4. Plan: the pre-flight

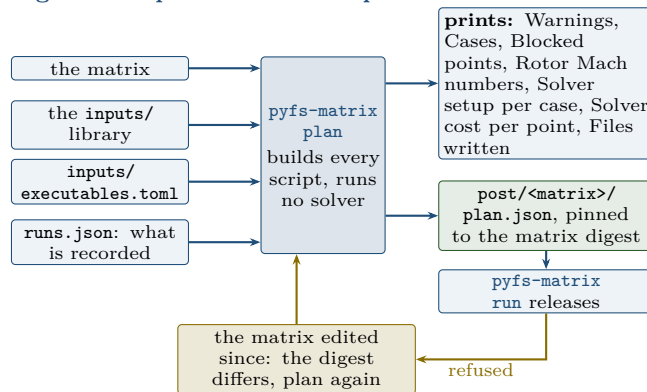
The plan binds every row to the library, builds every script in a dry run and says READY or BLOCKED with the reason, spending no solver time. It is mandatory: it writes the receipt a run asks for.



What it is for

Always plan before you run: a blocked point costs a message, a bad run costs the license. The plan prints its answer in blocks, writes `post/<matrix>/plan.json` pinned to the digest of the matrix it read, and a run refuses a matrix edited since. Its warnings refuse nothing; read them before you run. Exit code 1 when a point is blocked, 0 otherwise.

Figure: the plan and its receipt



A typical sequence

```
pyfs-matrix plan study.fs --workspace . --fs-version 26.123
pyfs-matrix plan study.fs --workspace . --cost
pyfs-matrix run study.fs --workspace . --fs-version 26.123
```

The first plan names what blocks; fix it and plan again until every point is READY. The second adds the cost table. The run gives the plan's values again.

The output blocks

Warnings go to standard error; everything else to standard output. **Cases** counts the points ready, blocked and already recorded.

Blocked points names each one and why. **Rotor Mach numbers:** tip and helical Mach per rotor and point. **Solver setup per case:** model, boundary layer, viscous coupling, aliases. **Files written:** the receipt and any guide. `logs/activity.log` keeps every line.

A grouped plan: several points, one solver

A grouped run solves several points of a polar, or several polars, in ONE solver instance. `--polar-sweep` plans one job per polar; `--batch N` plans N jobs of whole polars, grouped by processors, build, kind (steady or unsteady) and user actions, the longest as short as it can be (at least one job per group, warned). An acoustic polar is a job of its own. The plan prints one line per job (batch id sims cpus points estimate walltime source) and every polar *left out of the grouping* with its reason, and records the split in `plan.json`; the run states the same option.

Commands

pyfs-matrix plan <matrix>

Pre-flight every point; no solver runs. Its alias is **pyfs-matrix inspect-setups**. The artifact options are on stage 3.

`--workspace DIR` the campaign root with `inputs/` (default: current folder).
`--name NAME` the campaign name (default: the workspace folder's name).
`--fs-version V` the build for rows whose FS_BUILD cell is empty, e.g. 26.123.
`--fs-exe PATH` an executable override; mandatory for a row whose FS_BUILD says MANUAL, otherwise the build resolves through `inputs/executables.toml`.
`--workflow CODE=NAME` a legacy script code to a run type (repeatable), the mapping **pyfs-matrix run** takes.
`--sims SIM [SIM ...]` plan only these simulations, space separated (2031 2032); the matrix is not edited.
`--points POINT [POINT ...]` with `--sims`: only these points, by the point name the plan prints.
`--recipe CODE=MODULE:FUNCTION` a LEGACY row's recipe (repeatable).
`--point-name TPL` the naming template; default {polar}, which is P<sim>-<point name>; also {point} {alpha} {beta} {mach} {advance_ratio} {sim} {campaign}.
`--cost` table each point's expected cost: mesh size, marked trailing edges, farfield layers, viscous coupling, steady or unsteady, time iterations, processors, and an EXPECTED wall time fitted on this workspace's own runs (unknown without a comparable run).
`--cost-file NAME` price by `inputs/costs/NAME.toml` (`--cost`, grouped estimate) when several exist.
`--polar-sweep` one job per polar; `--batch N`: N jobs (left).
`--hpc NAME` `inputs/hpc/NAME.toml`, when several exist.
`--inflow-fft` for each `qsteady_rotor` wheel point in a custom inflow: the harmonics one blade meets, `k_eff`, `n_max` and the suggested `PASSAGE_POSITIONS`; warns when the row states fewer.
`--update-ids` (also `--updateIDs`) REWRITE this matrix so no POL repeats one used elsewhere in the workspace; written before the plan runs, and kept if the plan then refuses.
`--accept-unregistered-build` allow an installed build other than the registered one; its compatibility is your responsibility and every record says so.
`--verbose` every warning in full, and the started and finished lines of each continuation.

Reading the cost table

The expected wall time is an extrapolation from this workspace's recorded runs, and the table carries the number of samples behind it. A new workspace reads **unknown**: run a few points first, then plan again with `--cost`. A cost file, one per machine, replaces that fit; only a synthetic `examples/costs/c000.toml` ships.

WALLTIME and BEST in a grouped job

A job asks for the sum of its points' WALLTIME cells (each budgets ONE datapoint), capped at the profile's `max_walltime` with a warning (a larger N helps); a cell above it is refused. **BEST** prices the job: estimate

times 1.25 plus `walltime_margin_s`, up to the minute; an unknown point takes `max_walltime`, warned, or is refused without one. A plan without `--batch` or `--polar-sweep` refuses **BEST**.

Files

R `<matrix>.fs` from either home (stage 2).
R `inputs/` every artifact a row names, and `inputs/executables.toml` for each FS_BUILD.
R `inputs/costs/c<NNN>.toml` the machine's cost file.
R `inputs/hpc/<NAME>.toml` its `max_walltime`.
R `runs.json` which points are already recorded.
W `post/<matrix>/plan.json` the receipt **pyfs-matrix run** asks for, pinned to the matrix digest; grouped: its jobs, polars left out.
W `logs/activity.log` every line the plan printed, and the detail `--verbose` adds.
W `<matrix>.fs` rewritten only by `--update-ids`, each change printed.

Common errors

! A point is **BLOCKED**, with the reason under *Blocked points*.
Fix: fix the row or the input it names, then plan again; a blocked point never runs.
! The build installed is not the one registered for the row's version.
Fix: refused: install the registered build, or pass `--accept-unregistered-build`, recorded in `plan.json` and on every record.
! A row's FS_BUILD cell is empty and no fallback is given.
Fix: fill the cell, or pass `--fs-version`.
! A run type refuses the build: outside the range it covers.
Fix: the refusal names the builds it covers and the commands that decided it: choose one of them in FS_BUILD.
! A POL repeats one stated in another matrix or earlier in this file.
Fix: `--update-ids` rennumbers the repeated rows; a row whose POL already has runs of this matrix is refused rather than moved.
! The plan warns: helical Mach of 1 or more on some points.
Fix: a warning, not a refusal: check the rotor speed of those points.
! The plan lists points a **RESTART** key does not continue.
Fix: **FINISH_PENDING** on a **CONVERGED** run is refused with the reason; use **ADDITIONAL_REVS** or **ADDITIONAL_ITERS**.
! The plan warns that an unsteady row's geometry carries saved solver actions.
Fix: clean it with **pyfs-matrix inventory --clean** (stage 8); only a grouped plan blocks on it.
! A polar is *left out of the grouping*.
Fix: named, never refused: **LEGACY**, **RESTART**, **COLD_START** false and coupled **steady/qsteady_rotor** rows; unsteady rows on a build without the action command; polars that do not splice or are recorded. Run them point by point.

Where to read more

The documentation page *Planning a campaign and what it costs*.
Copyright (c) 2026 Geovana Neves, CC BY 4.0.

<https://github.com/nevesgeovana/pyflightstream>. Every command and option: `<tool> <cmd> -h`, or pages 1 and 2 of this cheatsheet.

5. Run: on this machine or on the cluster

The run builds every active point's script and executes it: on Windows locally, on Linux with a submission profile by submitting each point, or each grouped job, to the scheduler. Every point lands in the records with exactly one status.

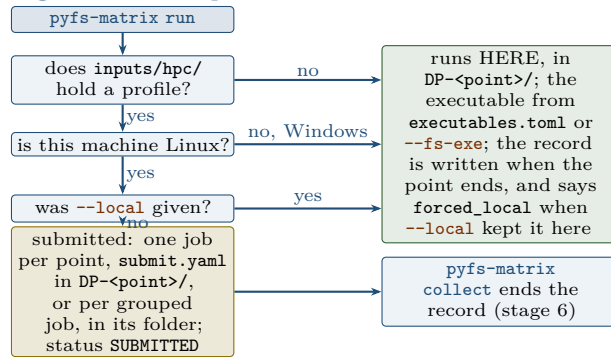
pyflightstream **0.38.0** the campaign workflow by stage, stage 5 of 8



What it is for

A run needs a current plan. Each point runs in its own datapoint folder, `sims/sim_<POL>/datapoints/DP-<point>/`, where its outputs are filed; a steady row is one job over all its points, run in the simulation folder. A local point is recorded when it ends; a submitted point is recorded `SUBMITTED` and finished by `pyfs-matrix collect` (stage 6). There is no separate submit command.

Figure: where a point runs



A session on the cluster

```
pyfs-matrix plan study.fs --workspace .
pyfs-matrix run study.fs --workspace .
pyfs-matrix collect --workspace . --watch
```

On Linux with a profile the run submits every point and returns; collect then sweeps until nothing is outstanding. On Windows the same run executes the points here and needs no collect.

The submission profile

One `inputs/hpc/<NAME>.toml` per cluster; one needs no option, several need `--hpc NAME` (else refused rather than guessed): the descriptor and its fields, the submit command, `[builds]` (the scheduler's name of each build, e.g. "26.123" = "26.1"), `[log]` (`export_log`, `native_log`, `job_end_files`) and `max_walltime`.

A grouped run: one solver, many points

`--batch N` or `--polar-sweep` runs the jobs of the plan's receipt, refusing one made for another mode, N or selection, one with no job, or one whose job folders hold files: plan again. A polar's job is `sims/sim_<POL>/FULL-POLAR.txt`; a batch's is `BATCH-<first>-<last>.txt` in `sims/batch/<matrix>_b<ID>/`, the ID the package's. The job's clock replaces each point's `WALLTIME`. Every point is recorded `SUBMITTED`, also when the job runs here (one instance, one license, one job at a time): collect, then post. Not with `--force-rerun`, `--force-rerun-all` or `--sweep-csv`.

Commands

`pyfs-matrix run <matrix>`

Run every active point and write the sweep table.

- `--resume` skip points already recorded, so a grown matrix runs only its new points; without it a recorded point refuses.
- `--force-rerun POINT REDO` a recorded point, by point name, full run id or a steady row's job id (repeatable): the records and outputs are archived first, nothing is deleted.
- `--force-rerun-all REDO` every recorded point; the count of points and jobs, the licenses it spends, prints first.
- `--sims SIM [SIM ...]` run only these simulations, space separated (2031 2032); with `--force-rerun-all`, redo only these.
- `--points POINT [POINT ...]` with `--sims`: run only these points, by the point name the plan prints; refused beside `--force-rerun-all`.
- `--local` run on THIS machine instead of submitting: a Linux workstation, or a smoke test.
- `--polar-sweep` run the receipt's jobs, one per polar; `--batch N`: its N jobs (left).
- `--hpc NAME inputs/hpc/NAME.toml`, when several exist.
- `--progress-every N` a local unsteady point reports every N steps (default 10; 0 is silent).
- `--sweep-csv FILE` the sweep table (default `post/<matrix>/campaign_sweep.csv`).
- `--workflow CODE=NAME` a legacy script code to a run type: `steady`, `unsteady`, `unsteady_rotor`, `qsteady_rotor` (repeatable).
- `--workspace DIR --name NAME, --fs-version V, --fs-exe PATH, --recipe CODE=MODULE:FUNCTION, --point-name TPL`: as in plan; give the plan's values.
- `--ignore-missing-families [true|false]` and `--no-ignore-missing-families`: as in plan.
- `--accept-unregistered-build` as in plan; every record says it was used and carries the build the solver printed.
- `--pproc-warnings` print the post-processing warnings, grouped.
- `--verbose` every warning in full, one line per repeated item.

Redoing a point

A correction that does not change a point's name (a pproc, a geometry, a solver variable, a wall clock) keeps its identity: plan again, then `--force-rerun` it. A correction to `FLIGHT_CONDITION` changes the name: it is a new point, and `--resume` runs it beside the old record.

Files

- R** `post/<matrix>/plan.json` must be current: the digest of the matrix it read.
- R** `inputs/executables.toml` each `FS_BUILD` to its executable.
- R** `inputs/hpc/<NAME>.toml` the submission profile, when the workspace has one.
- W** `sims/sim_<POL>/scripts/` the generated solver scripts, `P<sim>-<point>.txt`.
- W** `sims/sim_<POL>/datapoints/DP-<point>/` the point's outputs, its descriptor when submitted, its wall-clock state.
- W** `runs.json` one record per point or steady job; `runs.json.lock` while it is written.
- W** `post/<matrix>/campaign_sweep.csv` the sweep table of this matrix.
- W** `archive/runs-<stamp>.json` the manifest before a forced re-run; each redone point's outputs go to its own `archive/<stamp>/`.

Common errors

! *no plan for this matrix, and since v0.17.0 a run needs one.*

Fix: run `pyfs-matrix plan` with the same matrix first.

! The matrix was edited after the plan.

Fix: refused until planned again: the receipt is pinned to the matrix digest.

! A point is already recorded and refuses.

Fix: the refusal counts the points recorded and those `--resume` would run, and prints the command that continues; `--force-rerun` redoes a point. `--resume` and `--force-rerun` together are refused.

! `--sims` names an id, or `--points` a point, the matrix does not carry; or `--points` comes without `--sims`.

Fix: refused before anything runs, naming the ids or points that exist.

! `--force-rerun` names a point no record carries.

Fix: refused: copy the name or run id the refusal printed.

! `--force-rerun-all` is refused.

Fix: beside `--resume`, `--force-rerun` or `--points`, for a `--sims` id the matrix does not carry, or when nothing of the selection is recorded.

! Several profiles in `inputs/hpc/`, no `--hpc`.

Fix: refused rather than guessed: name one.

! The profile has no `[builds]` line for a row's build.

Fix: refused before any point is submitted: add the line.

! A `WALLTIME` cell written as a bare number is refused by name.

Fix: write it with its unit (stage 2).

! `--resume` on a simulation a rebuild refused (stage 8).

Fix: never: without a record the resume runs it again. Settle the rebuild first.

Copyright (c) 2026 Geovana Neves, CC BY 4.0.

<https://github.com/nevesgeovana/pyflightstream>. Every command and option: `<tool> <cmd> -h`, or pages 1 and 2 of this cheatsheet.

6. Collect and the run records

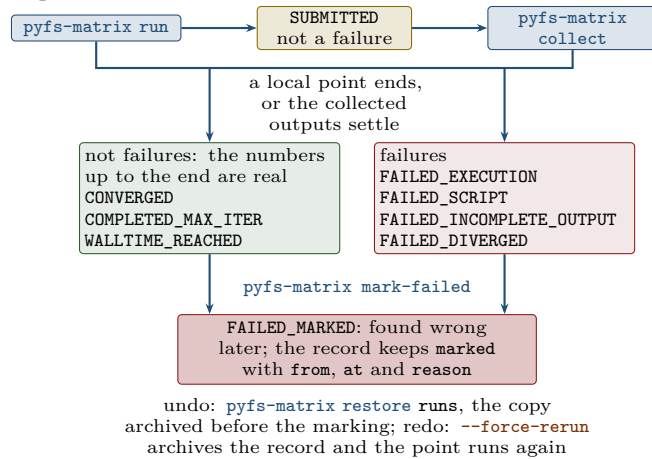
A submitted point comes back when its outputs land: collect waits for them, files them, judges the run and updates its record. The records, `runs.json`, are the study's evidence; sync keeps a workstation and a cluster in agreement.



What it is for

For every SUBMITTED record, `pyfs-matrix collect` waits until the declared outputs are present and SETTLED (the same size and time at two observations), collects them, assesses the run, writes its terminal status, then posts (stage 7). A local run needs no collect. Every executed point lands in exactly one status: a silently skipped point is impossible.

Figure: the life of a run record



What a record carries

Which run it was (`run_id`, `point_name`, `sweep_name`), which build ran it, the hashes of the script and the inputs, how the solver was called (`executor`), where it ran (`working_dir`), how it ended (`status`), how long it took and which files it produced. Nothing is inferred from a folder name afterwards.

Read the recorded evidence. `pyfs-matrix show 9101_1` prints the outcome, evidence tails, logs, chain, identity and inputs (coupled: the FSI record, last exchange row); a simulation id lists its points and failed cards, or add `--point NAME`. `pyfs-matrix log` reads activity oldest first: `--sims IDS`, `--run ID`, `--stage NAME`, `--since 2h` (or an ISO time), `--problems`, `--open`. `--post MATRIX` groups saved post warnings by category, family and message shape with a count and example; `--sims IDS` narrows the groups. `--storage` reads storage calls as recorded; `--stage` selects their action. `pyfs-matrix trace post/demo/polars/polar.csv` follows the products index to run identities, digests and provenance; `--run ID` prints a run's provenance tree. All three accept `--workspace`, `--runs NAME`, `--json`, `--csv`, and run ids or aliases where a run is named. `pyfs-matrix status --additional` prints the additional register without deriving fields. They read running batch folders and compacted simulations in place, write nothing, never ask the scheduler.

`COMPLETED_MAX_ITER` and `WALLTIME_REACHED` are not failures (the numbers up to the cap are real); a reader asking whether a status begins with `FAILED` treats `FAILED_MARKED` as one. A forced re-run archives the record and runs the point again.

Commands

`pyfs-matrix collect`

Collect the outputs of every SUBMITTED record when they land, then post.

- `--watch` keep sweeping until nothing is outstanding (default: one sweep).
- `--discard-walltime` mark each `WALLTIME_REACHED` point `FAILED_MARKED` before post, outputs kept: the next grouped plan (stage 4) runs it again from the start; a default-mode plan needs a forced re-run (stage 5).
- `--hpc NAME inputs/hpc/NAME.toml`, when several exist.
- `--watch-interval S` seconds between sweeps under watch (default 60).
- `--interval S` seconds between the two observations that decide settled (default 2).
- `--rounds N` stop a watch after N sweeps.
- `--sims IDS` only the SUBMITTED records of these simulations (2006,2007), and their post.
- `--no-post` collect only, do not rebuild the products.
- `--check-frozen REFUSE`, not warn, averages touched by a frozen solve or an unread log block.
- `--workspace DIR --runs NAME` another manifest; `--pproc-warnings`;
- `--verbose`.

Tip: `nohup pyfs-matrix collect --watch > logs/collect.out 2>&1 &`

`pyfs-matrix sync {runs|post|fsm|all}`

Run on the MAIN workspace: bring records and results from the others named in `inputs/sync-workspaces.toml`. Levels are cumulative.

- `--apply` change files (default: preview).
- `--from NAME` one workspace (default: every non-main one).
- `--prefer-other` on a record conflict take the other's (default: main wins, unless main's is still SUBMITTED).
- `--overwrite` on a file conflict archive main's copy to `archive/sync-<stamp>/` and take the other's.
- `--restore` with `--apply`: rebuild the records of the `sims/` folders no record carries.
- `--include-archives` also bring folders named `archive` (skipped by default).
- `--workspace DIR --runs NAME`: merge into another manifest.

Levels: `runs` the records with each simulation's scripts and logs; `post` adds `post/`; `fsm` adds each datapoint's saved simulation; `all` adds the rest of `sims/`. A matrix is owned by one workspace, and the owner's copy wins.

Files

- R** `sims/sim_<POL>/datapoints/DP-<point>/` the outputs and the solver log a SUBMITTED point waits for.
- R** `sims/batch/<matrix>_b<ID>/` a batch job's folder: its script, descriptor, log, end files and, while it runs, its simulations.
- R** `inputs/hpc/<NAME>.toml [log]` `job_end_files`: the files a scheduler writes when a job ends.
- W** `runs.json` the records: `run_id`, `status`, the build, the hashes of script and inputs, `working_dir`, times; `runs.json.lock` while written.
- R** `inputs/sync-workspaces.toml` main, each workspace's path and the matrices it owns.
- W** `archive/runs-<stamp>.json` the manifest before a sync rewrites it.
- W** `storage_management.json` one entry per sync source.

Collecting a grouped job

Collect reads the job's descriptor, log and end files in its folder and completes each point as soon as its files settle, while the job still runs. Until a batch ends it copies each finished simulation to `sims/` and leaves the batch folder alone; once it ends, it moves them. It cuts the job's log into one log per point. A point that failed without stopping the instance does not stop the job; the points a stopped job never started are named, and the next grouped run takes them again. A point the job's clock stopped is `WALLTIME_REACHED`: see `--discard-walltime`.

Common errors

- !** A job died before its log: the point stays SUBMITTED for ever.
Fix: name the scheduler's end-of-job files in the profile, `[log] job_end_files = ["FTS{sim}.o*", ...]`: when they all exist and the log does not, collect records `FAILED_EXECUTION` with their last 20 lines.
- !** Averages touched by a frozen solve.
Fix: by default the products are written and `post.log` warns; `--check-frozen` refuses them instead.
- !** The sync is refused: `runs.json.lock` in main.
Fix: a run is writing here: wait for it. A lock in a source skips that source, recorded.
- !** A stem is declared by two workspaces, or a matrix by none.
Fix: refused: each matrix belongs to exactly one workspace in `inputs/sync-workspaces.toml`.
- !** `--restore` with `--runs`.
Fix: refused: the restore appends to `runs.json` only.
- !** A `runs.json` that holds `FAILED_MARKED` is unreadable by an older release.
Fix: keep 0.33.0 or later on every machine sync joins, or restore the archived copy before going back.

7. Post and products

The post builds the product tables from the records, with no solver and no seat: polars, sections, probe tables, per-step series and sampled fields, each with its provenance. A rebuild archives what it replaces.



What it is for

`pyfs-matrix run` and `pyfs-matrix collect` post on their own; `pyfs-matrix post` rebuilds by hand, from `runs.json`, under `post/<matrix>/`, the folder named after the matrix stem. Without a matrix it posts every matrix the manifest names. Nothing is refused and nothing lost: what is there moves to `archive/<stamp>/` first. Exit codes: 0 done; 2 refused, nothing written; 3 with `--strict`, a product skipped by design.

Figure: the products of one matrix

```
post/<matrix>/
├── plan.json the plan receipt (stage 4)
├── campaign_sweep.csv the sweep table
├── products.json every file, its run ids, skips
├── post.log post.log.json warnings and refusals
├── polars/ one polar table per group
├── sections/ <point>_sections.csv
├── probes/ <point>_plots.csv, <point>_probes.csv
├── series/ per-step series of an unsteady point
├── fields/ sampled velocity fields
│   ├── <stem>.vtk VTK vertex cloud
│   ├── <stem>.dat Tecplot
│   ├── <stem>.inflow.dat reusable inflow, SI
│   └── <file>.provenance.json beside each field file
├── archive/<stamp>/ what each rebuild replaced
└── post/<matrix>@sims/ products of --from-sims
```

Sampled fields and their provenance

Positions are in REFERENCE coordinates and meters, velocity absolute in REFERENCE components and m/s, one file per unsteady STEP. The provenance names the original frame, its recorded motion, the STEP, the coordinate conversion, the source-file hash and the velocity evidence. The convention was measured on one 26.124 build; a field of another 26.124 build is written with one warning per point and, in its `products.json` entry, `velocity_convention` with proven false. To reuse an inflow file, state `FREESTREAM_UNITS:SI` on the row.

Commands

`pyfs-matrix post [matrix]`

Rebuild the post-processed products from the records, with no solver.

- `--workspace DIR` the campaign root (default: current folder).
- `--sims IDS` only these simulations' products, in place; the super files stay as the last whole post wrote them.
- `--runs NAME` read another manifest, e.g. one a rebuild wrote.
- `--from-sims` ignore `runs.json`: records assembled in memory from `sims/`, for runs made outside the package; products to `post/<matrix>@sims/`.
- `--steps-per-revolution N` with `--from-sims`: for a `LAST_REVS_AVG` window.
- `--additional-pproc` first reopen the final `.fsm` of each recorded point whose row states `ADDITIONAL_PPROC`, with no solve, and extract that pproc; one solver launch per point; 26.124 only.
- `--fs-version V --fs-exe PATH, --local, --recipe CODE=MODULE:FUNCTION`: for the additional post, as in run.
- `--diagnostics` print the recorded post diagnostics as Markdown, change nothing.
- `--strict` exit 3 when a product was skipped by design, after writing everything.
- `--check-frozen` refuse instead of warn (stage 6).
- `--force-overwrite` DESTROY products instead of archiving; asks first, and `--yes` answers.
- `--pproc-warnings` grouped post-processing warnings; `--verbose` every one in full.

What the tables hold

Every table opens with POL. A polar table has one row per point: the reference block, J, and the body, stability and wind axis coefficients. The probe table opens with PROBE, X, Y, Z, FRAME, STEP; a cell the package cannot fill reads NA, never blank.

Files

- R** `runs.json` the records the products derive from.
- R** `sims/sim_<POL>/datapoints/DP-<point>/` each point's collected exports.
- R** `sims/sim_<POL>/profiles/<POL>_probe_points.csv` where each probe was, recorded at emission.
- W** `post/<matrix>/` the products, `products.json`, `post.log`, `post.log.json`.
- W** `post/<matrix>/archive/<stamp>/` what the rebuild replaced, one folder per rebuild.
- W** `additional.json` the record of the additional post; its files go to `datapoints/DP-<point>/additional/<pid>/`.

Common errors

- !** `--yes` without `--force-overwrite`.
Fix: refused: it answers a question only that option asks.
- !** A product is skipped by design (a polar under sideslip).
Fix: named on standard error and under `skipped` in `products.json`; `--diagnostics` prints why. Exit 0, or 3 with `--strict`.
- !** A field of another solver version, unit or export family.
Fix: refused: *native velocity convention has no evidence for this export/build/unit*. Measure the convention on that build first.
- !** The matrix is in both homes with different bytes.
Fix: the post warns naming both and builds from the run records; keep one copy.
- !** The additional post on a workspace that submits from here.
Fix: refused without `--local`: only the local half is built.
- !** A point's `.fsm` does not hash as its record says.
Fix: skipped by name: the additional post reads only the file the run saved.
- !** An old product is gone after a rebuild.
Fix: it is in `post/<matrix>/archive/<stamp>/`; `pyfs-matrix restore` products brings back `products.json`.

Where to read more

Guide 5 and the page *Post-processing definitions*, the definition of record of every product; the pages *Sampled fields* and *Extracting more from a finished point*.

Copyright (c) 2026 Geovana Neves, CC BY 4.0.

<https://github.com/nevesgeovana/pyflightstream>. Every command and option: `<tool> <cmd> -h`, or pages 1 and 2 of this cheatsheet.

8. Maintenance: space, records and saved geometries

A workspace grows and records go wrong. These commands measure and free space, delete simulations, mark a run failed after the fact, restore or rebuild the records, and clean a saved geometry. Each previews until `--apply`.

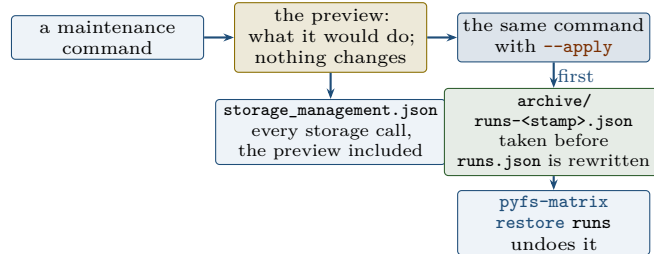
pyflightstream **0.38.0** the campaign workflow by stage, stage 8 of 8



What it is for

Run the preview, read it, then apply. Every storage call, the preview included, is recorded in `storage_management.json`, and every rewrite of `runs.json` archives the previous copy first, so `pyfs-matrix restore` can undo it.

Figure: preview, apply, undo



A storage recipe, `inputs/management/m<id>.toml`, runs its tables in this order:

1. `prune_step_exports`: keep only the last step of each per-step export
2. `compact_sims`: zip a simulation folder, restored automatically when needed
3. `delete_extensions`: delete the files of the named extensions
4. `post_archives`: compact or delete the post's archived folders

never touched by a recipe: `inputs/`, a matrix, `storage_management.json`, a saved `.fsm`, `scripts/`, every log, every file a record names or hashes

pyfs-matrix inventory <geometry>

Clean a saved simulation that keeps a previous run's unsteady solver actions.

- `--clean` remove them, keeping `<file>.bak-<stamp>`; a 26.120 file in meters is also reset to a fresh import, meshes and boundary conditions kept.
- `--overwrite` also rewrite the existing sidecar.

Commands

pyfs-matrix history <simulation or point>

Every present and archived record, continuation link and archive stamp; reads batch folders and compacted simulations in place.
`--workspace DIR` workspace root; `--runs NAME` selects another manifest and its archives.
`--json`, `--csv` the same rows as machine data; writes nothing.

pyfs-matrix diff <run-a> <run-b>

Compare versions, digests, solver flags, flight condition and outcome. A run id with `@<archive stamp>` selects an archived copy.
`--workspace DIR` workspace root; `--runs NAME` selects another manifest.
`--json`, `--csv` the same changes as machine data; script changes name lines and inputs.

pyfs-matrix space-in-use

Sizes on disk by top folder, by `sims/sim_*`, by extension; changes nothing.
`--top N` rows per grouping (default 15). `--workspace DIR`.

pyfs-matrix free-space <m-id>

Run the recipe `inputs/management/m<id>.toml`.
`--apply` change files (default: preview).
`--list` every path each step touches, its size and what happens to it.
`--runs NAME` also read this manifest; `--workspace DIR`.

pyfs-matrix delete-sims <ids>

Delete simulations (4001,2009), their products and records; a note row keeps the ids retired.
`--apply` change files (default: preview).
`--force` also a simulation whose run is still SUBMITTED.
`--matrix-products {points-only,regenerate}` a product that also holds other points: leave it, marked stale, or post it again without the deleted ones.
`--runs NAME` read and edit another manifest; `--workspace DIR`.

pyfs-matrix mark-failed

Every record of the named simulations becomes FAILED_MARKED, its old status kept under marked.
`--sims IDS` required: 2006,2007.
`--reason TEXT` why, recorded as given.
`--apply` write (default: preview); `--workspace DIR`.

pyfs-matrix mark-converged

The named points become CONVERGED by your reading, their old status kept under marked.
`--sims IDS` required: 2006 2007; `--points NAMES` only these points.
`--reason TEXT` required, recorded as given.
`--apply` write (default: preview); `--workspace DIR`.

pyfs-matrix restore {runs|storage|products|plan|additional}

Bring a records file back from the archive, byte for byte.
`--apply` write (default: preview); the current file is archived first.
`--stamp STAMP` the copy to restore (default: the newest).
`--matrix STEM` for products and plan, when several matrices have archives; `--workspace DIR`.

pyfs-matrix rebuild

Make run records again from `sims/`: each matrix row is rendered again and compared with the script that ran.

- `--apply` append the rebuilt records to `runs.json` (default: preview).
- `--out NAME` write them to another manifest instead, to compare.
- `--all-sims` every simulation folder, recorded or not; needs `--out`.
- `--sims IDS` only these: 4001,2009.
- `--matrix FILE` the matrix the simulations ran from.
- `--build-alias BUILD=ALIAS` the scheduler's name of a build then (repeatable).
- `--inputs-from FOLDER` another origin's `inputs/` for inputs changed after the run; `--workspace DIR`.

pyfs-workspace archive <root> <sim_id>

Zip one recorded simulation under `archive/` and remove its folder.

Files

- R** `inputs/management/m<id>.toml` the storage recipe.
- W** `storage_management.json` every storage call, `force` and the statuses included.
- W** `runs.json` rewritten by `pyfs-matrix delete-sims`, `pyfs-matrix mark-failed`, `pyfs-matrix rebuild`, `pyfs-matrix restore`.
- W** `archive/runs-<stamp>.json` the copy taken before each rewrite.
- W** `sims/sim_<POL>.zip` a simulation compacted by `compact_sims`, restored automatically when needed.
- W** `<file>.bak-<stamp>` the geometry before `--clean`.

Common errors

- ! *have a run still SUBMITTED; collect it first.*
Fix: run `pyfs-matrix collect`, or delete it anyway with `--force`.
- ! `--apply` refuses, naming a product shared with other simulations.
Fix: choose `--matrix-products points-only` or `regenerate`; `regenerate` is refused with `--runs`.
- ! A simulation id with no record (mark-failed).
Fix: refused by name before anything is written; a record already FAILED_MARKED is left as it is.
- ! The rebuild refuses an edited or drifted script, naming the input.
Fix: the inputs changed after the run: `--inputs-from` the folder they came from.
- ! The profile no longer maps an old build.
Fix: `--build-alias 26.123=26.12`.
- ! A POL no current matrix holds.
Fix: `--matrix` the file the simulations ran from.
- ! `--out` names `runs.json` or a file that exists.
Fix: refused before any work: choose a new name.
- ! The restore is refused while `runs.json.lock` exists.
Fix: a run, collect or sync is writing: wait for it.
- ! A cleaned geometry has a new hash.
Fix: runs staged before the clean keep theirs; a new campaign records the new one.

Copyright (c) 2026 Geovana Neves, CC BY 4.0.

<https://github.com/nevesgeovana/pyflightstream>. Every command and option: `<tool> <cmd> -h`, or pages 1 and 2 of this cheatsheet.